TECHCIRCUIT HACKATHON | ROUND 3 SUBMISSION

BrandGuard AI

Intelligent Brand Governance Platform

PROJECT NAME

BrandGuard AI

PROBLEM STATEMENT PROVIDER

Inkloom AI

TEAM NAME | SUBMISSION ROUND

ChipSmiths | Round 3

TEAM MEMBERS

1 Syeed Hussain S H

2 U Tamilarasu

3 Shyam G R

4 Sandep S

LIVE PROJECT

https://brandguard-ai-w20n.onrender.com

GITHUB REPOSITORY

https://github.com/syeedhussain-sh/BrandGuard-AI

Problem Statement and Solution

Checking brand content against approved guidelines, facts and review rules

THE PROBLEM

Brand content created by people and AI tools drifts from approved guidelines. Inconsistent messaging, outdated facts, unsupported promotional claims and incorrect terminology are usually found by manual proofreading, which depends on the reviewer’s knowledge and attention.

WHY IT MATTERS

Without a consistent check, a single unsupported claim can reach customers under a trusted brand name.

HOW BRANDGUARD AI HELPS

Check against guidelines

Compares content with the selected brand’s approved rules and terminology.

Identify inconsistent messaging

Flags off-brand terms, restricted phrases and cross-brand mismatches.

Flag unsupported claims

Checks claims against approved facts and marks what cannot be verified.

Suggest corrections

Proposes replacements and shows original and corrected copy side by side.

Support human review

Routes flagged content to reviewers who approve or reject it, with a logged reason.

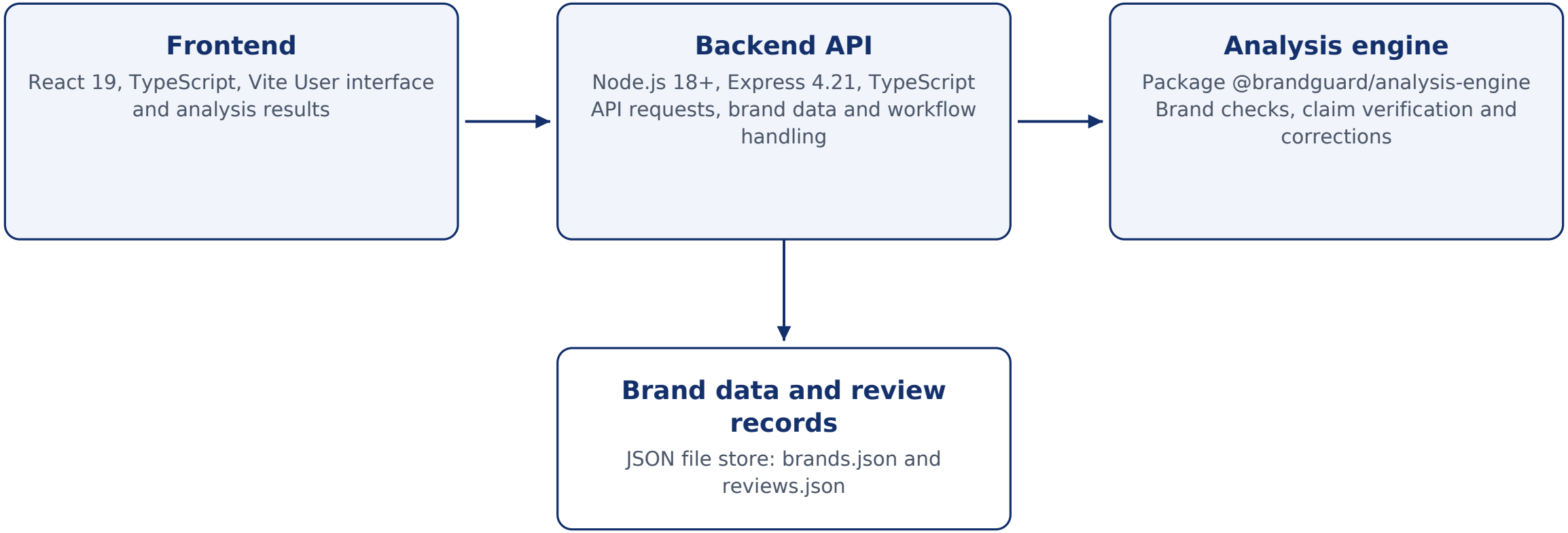
Final Product Overview and Features

Features verified in the deployed application

Feature	What the final version does
Brand selection	Active brand selector for configured brands (NovaTech, KFC) and generic brands (Tata Motors)
Brand guideline management	Active Brand Policy shows restricted phrases, glossary terms, claim checks and required terms
Content analysis	Audits submitted marketing copy against the selected brand
Terminology and capitalisation checks	Flags non-canonical names, e.g. “Nova Cloud”, and suggests “NovaTech Cloud Platform”
Cross-brand mismatch detection	Flags claims outside a brand’s business, e.g. a vehicle claim for KFC
Claim verification	Shows which claims are supported by an approved fact and which need verification
Issue severity and explanations	Issues grouped as High, Medium or Low, each with a policy explanation
Suggested corrections	Corrected copy, original copy and side-by-side comparison
Generic mode	Universal checks for brands without configured guidelines, labelled as general indicators
Human review and approval	Approve or Reject each analysis; rejection requires a reason
Audit and review history	Review log with analysis ID, brand, decision, reviewer, date and rejection notes

Technical Architecture

How the components work together in the final version



Frontend and backend communicate over HTTP (/api). The backend uses the analysis engine package for every audit.

API Contract, Data, Build and Deployment

Details that support the architecture diagram

1 API contract

JSON over HTTP. Endpoints: GET /api/health; GET /api/brands; GET and PUT /api/brands/:brandId; POST /api/analyze; POST /api/reviews; GET /api/reviews; GET /api/reviews/metrics; GET /api/reviews/analysis/:analysisId. Full contract: Brand Guard AI-Backend/API_CONTRACT.md.

2 Data storage

Atomic JSON file store in the backend: brands.json holds brand guidelines and approved facts; reviews.json holds approval decisions and their audit trail.

3 Build process

npm run build runs three steps: build:engine, build:frontend and build:backend. The latest report shows all builds completed successfully. Tests: npm test (engine and backend).

4 Deployment

Live at https://brandguard-ai-w20n.onrender.com, deployed on Render using render.yaml. A Dev Mock Mode allows interface testing without the backend. Vercel configuration (vercel.json) is also in the repository.

IMPROVEMENT 1 OF 3

Context-Aware Brand Analysis

Brand selection, industry context and better corrections

PROMISED IN ROUND 2

Evaluate content in its industry and product context, reducing incorrect brand mismatch warnings and preserving relevant product names.

IMPLEMENTED IN ROUND 3

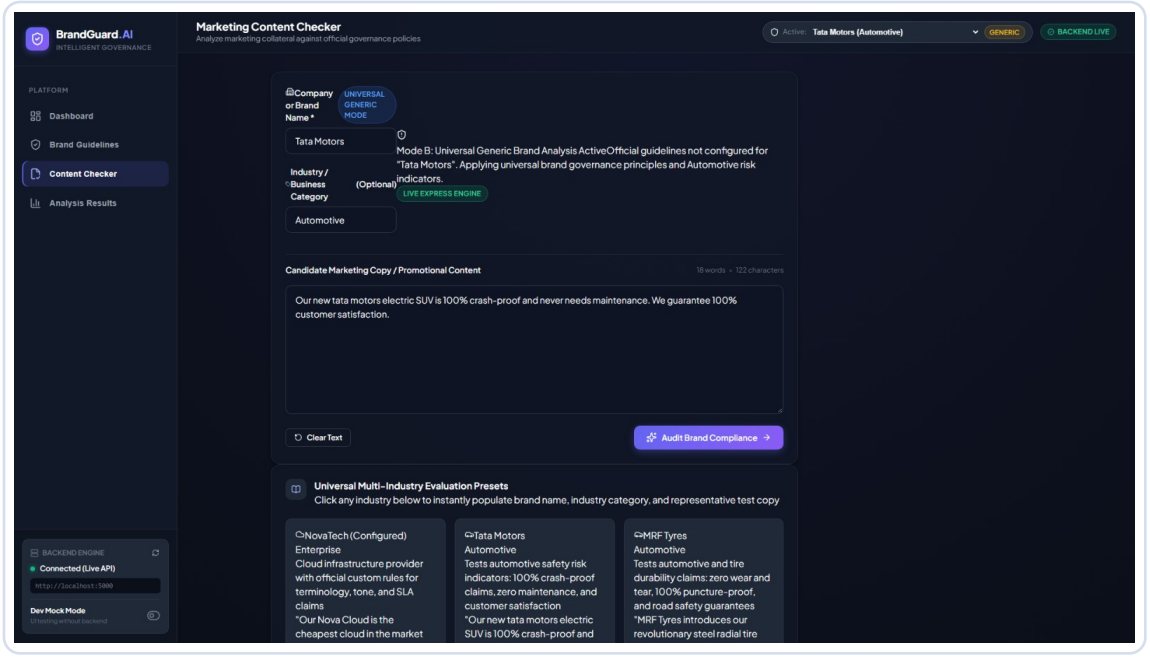
Brand selection with an industry category (e.g. Automotive, Food and beverages, Enterprise and Cloud AI). Brands without configured guidelines fall back to Universal Generic Mode, with the industry applied.

HOW IT WAS TESTED

94 analysis engine tests pass (per the repository README). Manual checks used NovaTech, KFC and Tata Motors sample copy.

STATUS AND LIMITATION

Completed for the tested cases. Mismatch detection depends on each brand's defined category and rules. Unconfigured brands receive general warnings only.



Brand selection with Universal Generic Mode: Tata Motors (Automotive) copy checked without a configured profile.

IMPROVEMENT 1 OF 3

Mismatch Detection and Corrections

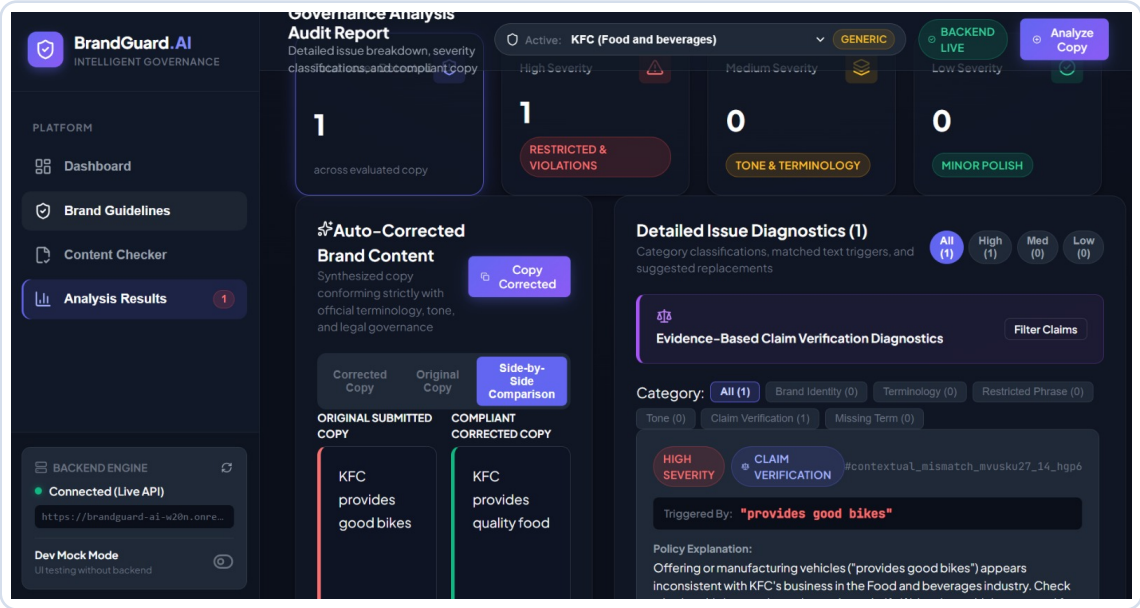
Examples of cross-brand mismatch, capitalisation and safe corrections

CROSS-BRAND MISMATCH

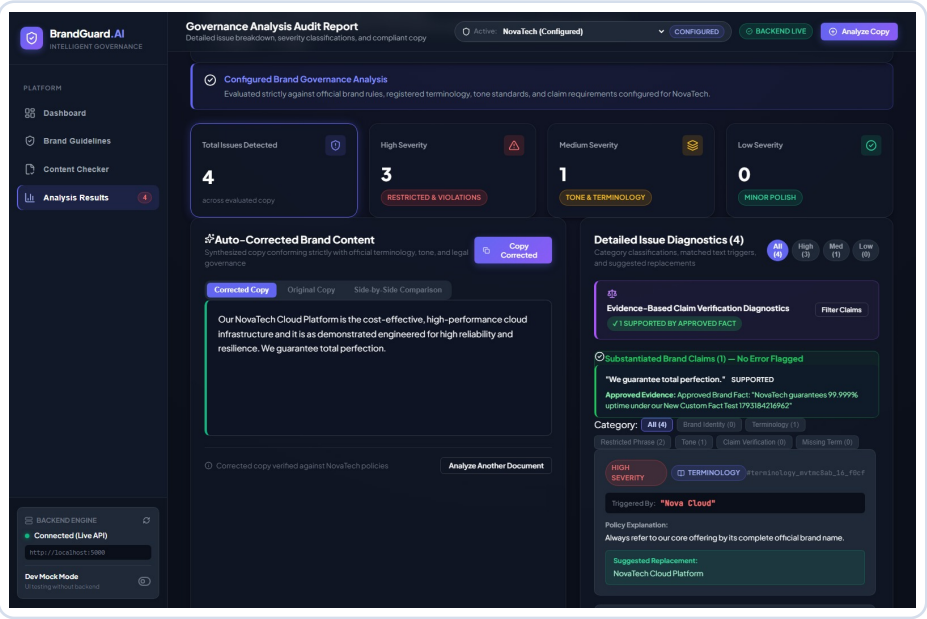
The KFC copy “KFC provides good bikes” was flagged as a contextual mismatch, because KFC operates in food and beverages. The corrected copy reads “KFC provides quality food”.

CAPITALISATION AND CANONICAL NAMES

“Nova Cloud” is flagged and replaced with the canonical name “NovaTech Cloud Platform”. Safe corrections are kept alongside issues that still need human review.



Cross-brand mismatch flagged in the audit report for KFC copy.



Capitalisation correction shown in the corrected copy.

IMPROVEMENT 2 OF 3

Evidence-Based Claim Verification

Approved facts, supported claims and claims flagged for verification

PROMISED IN ROUND 2

Distinguish unsupported claims from verified information, and give evidence or a clear explanation for claims that require verification.

IMPLEMENTED IN ROUND 3

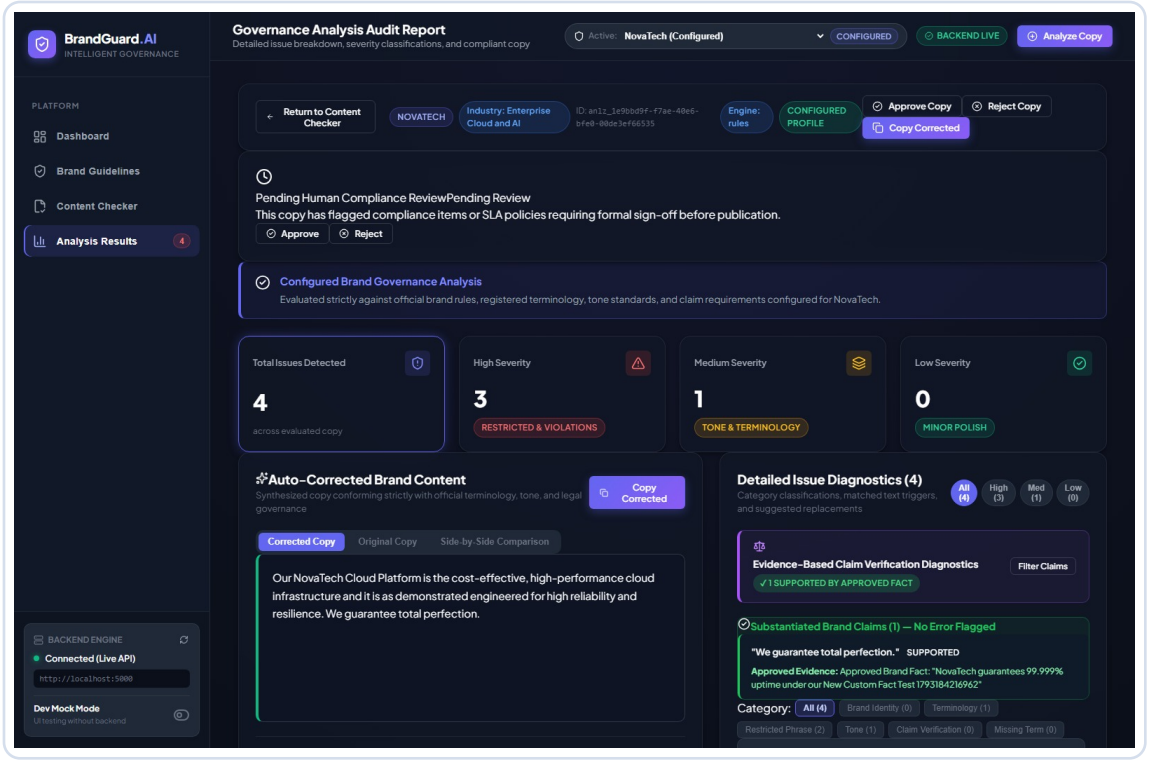
An Evidence-Based Claim Verification panel. Claims matched to an approved fact are listed as SUPPORTED with the fact cited. Claims without a match are flagged and routed to human compliance review.

HOW IT WAS TESTED

Engine tests pass. Manual tests used approved and unsupported claims.

STATUS AND LIMITATION

Partially complete. In the test copy, “We guarantee total perfection” is matched to an approved uptime fact (99.999%). That match is too lenient: an uptime fact does not establish “total perfection”. Tighter evidence matching is needed.



Pending human compliance review, with the claim verification panel showing a supported claim.

IMPROVEMENT 3 OF 3

Brand Guidelines and Approval Workflow

Guideline management and the human review and approval process

PROMISED IN ROUND 2

Let users define brand rules, review flagged content and approve content before publication.

IMPLEMENTED IN ROUND 3

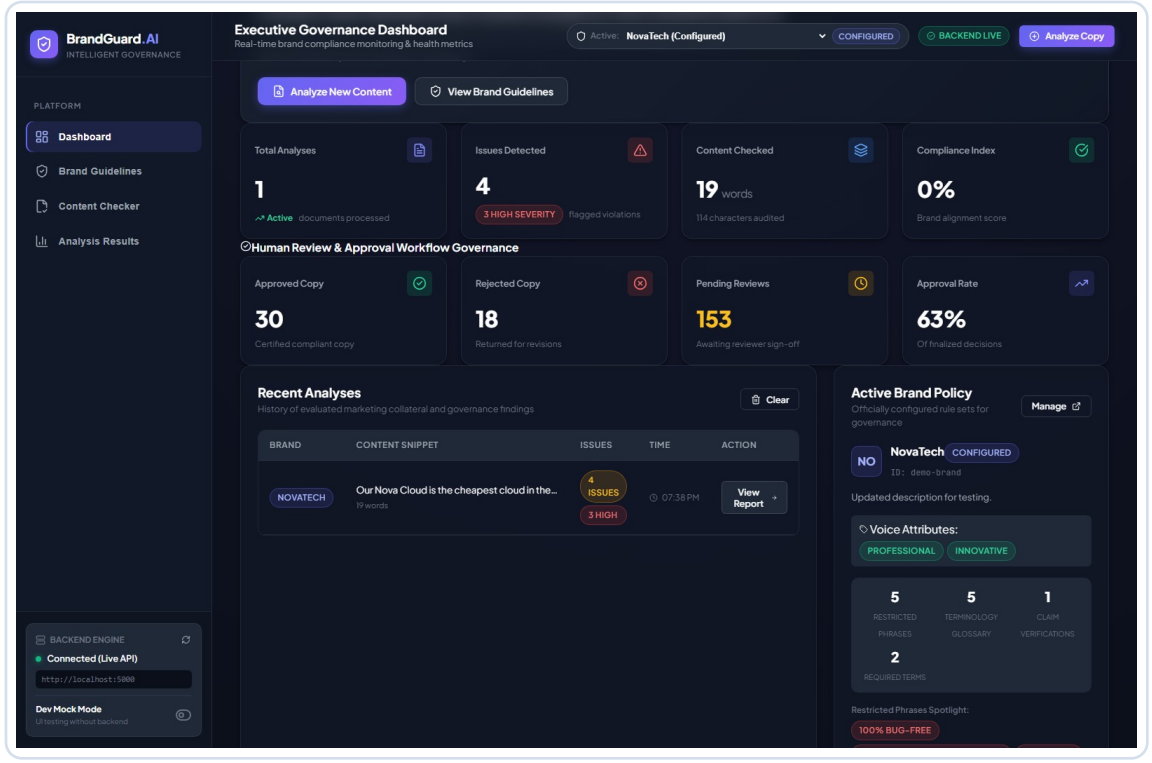
Configured brand profiles with an Active Brand Policy summary. A Human Review and Approval panel shows approved, rejected, pending and approval rate. Each report has Approve and Reject actions.

HOW IT WAS TESTED

Manual approval workflow tests, and checking that the dashboard reflects recorded decisions.

STATUS AND LIMITATION

Partially complete. The dashboard counts in this screenshot include test entries (e.g. 153 pending) and do not yet reconcile with the review log. They should not be read as production metrics.



Executive governance dashboard: review workflow counts and the active brand policy.

IMPROVEMENT 3 OF 3

Approval Process and Verification Status

How a submission moves from analysis to a logged decision

1. **Analyse**
   Content is audited and flagged issues are listed.

2. **Pending review**
   The report shows “Pending Human Compliance Review”.

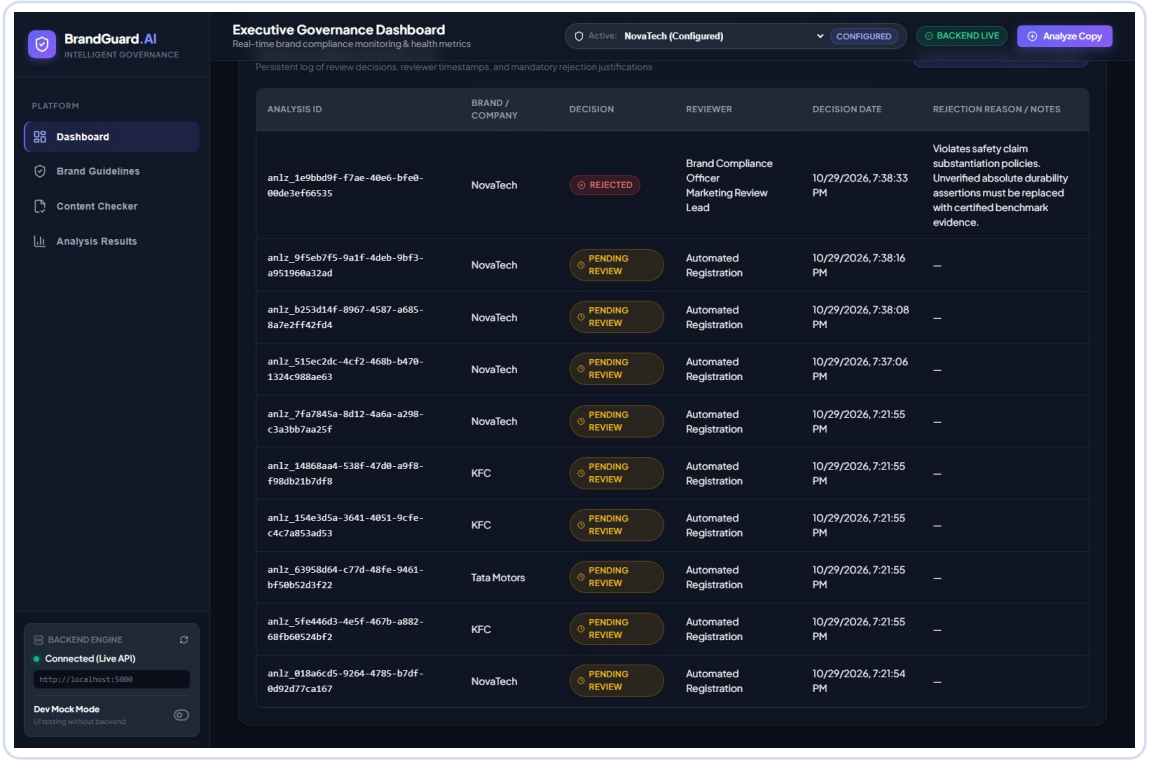
3. **Approve or reject**
   Reject requires a reason. Both decisions are logged.

4. **Review log**
   Analysis ID, brand, decision, reviewer, date and notes are recorded.

APPROVAL WORKFLOW: VERIFICATION STATUS

The Approve and Reject actions were tested on a mobile phone. Both actions completed successfully, and the resulting decisions appeared in the review log.

The previously observed approval timeout on another device has not yet been independently verified as resolved. Further cross-device testing is required before confirming that the issue is fixed.



Review log: each decision is recorded with analysis ID, brand, reviewer, date and rejection notes.

Testing and Reliability

Automated results documented in the repository, plus manual end-to-end checks

Test or build item	Reported result
Analysis engine tests	94 passed (12 files), per repository README
Backend API tests	55 passed (6 files), per repository README
Total automated tests	149 passed (18 files)
Frontend, backend, analysis engine and root builds	Successful (README: zero build errors)

The internal report figure of 172 tests (110 engine, 62 backend) does not match the repository README. Re-run npm test before submission.

MANUAL END-TO-END CHECKS

Check	Result
Approved and unsupported claims	Shown in screenshots; weak match noted in Improvement 2
Brand mismatch and capitalisation	Shown: KFC mismatch, “Nova Cloud” correction
Empty and invalid input	Automated: empty content returns 400 (per README). Manual: Not tested
Repeated actions	Automated: rapid consecutive audits (per README). Manual: Not tested
Refresh behaviour	Not tested
Approval workflow (mobile phone)	Approve and Reject working; decisions appear in the review log
Approval timeout on another device	Not verified: further cross-device testing required
Live deployment	Public site loads the application

Changes Made After Round 2

Summary of the Round 3 changes, including production consistency fixes

1. **Brand facts synchronised**
   Approved facts are now consistent between the source and production builds.

2. **Canonical capitalisation**
   Brand names are corrected to their canonical form, e.g. “Nova Cloud” to “NovaTech Cloud Platform”.

3. **Cross-brand mismatch**
   Better detection of claims outside a brand’s business, shown with the KFC example.

4. **Unsupported food-industry claims**
   Improved handling of unsupported claims in the food and beverages category.

5. **Safe corrections kept**
   Corrections that are safe to apply stay available when other issues need human review.

6. **Regression tests**
   Tests added to keep production and source behaviour consistent.

7. **Backend technology**
   The backend is built with Node.js, Express and TypeScript. The Round 2 plan named Python Flask; this changed.

Known Limitations and Future Development

Known problems are listed as they stand; future plans are not completed features

Known Limitations

- Claims that cannot be verified from available approved facts are flagged for human verification.
- Evidence matching can be too lenient: “We guarantee total perfection” is matched to an uptime fact.
- The approval timeout seen on another device is not yet verified as resolved. Only mobile-phone testing is complete.
- Dashboard counts include test entries and do not yet match the review log.
- Unconfigured brands receive general warnings only, not official findings.

Future Plans (not completed)

- More robust evidence sources for claim verification
- Additional brand guideline formats
- Better audit history and reporting
- Stronger automated end-to-end testing
- Improved review and approval controls
- Scalability and performance improvements

Links and Setup Instructions

Live website, repository and steps to run the project locally

LINKS AND REQUIREMENTS

Live website: https://brandguard-ai-w20n.onrender.com

GitHub repository: https://github.com/syeedhussain-sh/BrandGuard-AI

Prerequisites: Git, Node.js v18.0.0 or higher, npm v9.0.0 or higher (per README).

Demo access: no login is documented in the README. Not verified beyond the public site loading.

Environment variables: none are documented in the README. Not verified: confirm before deployment. Secret values are not included in this report or the repository.

Health check: http://localhost:5000/api/health

Windows: RUN_PRODUCTION.bat builds and starts the server, then opens the browser.

COMMANDS (CHECKED AGAINST ROOT package.json)

```
git clone https://github.com/syeedhussain-sh/BrandGuard-AI.git

cd BrandGuard-AI

npm install --prefix "Brandguard AI-Engine Analysis/BrandGuard-Analysis"

npm install --prefix "Brand Guard AI-Backend/backend"

npm install --prefix "Brandguard AI - Fe/frontend"

npm run build

npm start    # serves http://localhost:5000
```

Script names confirmed in package.json: build, start and test. The README’s “cd BrandGuard_AI” is replaced with the folder name created by git clone.